

Recap for approximation algorithm

- Greedy
- Local search
- Dynamic Programming
- Randomization + derandomization

} Basic tools

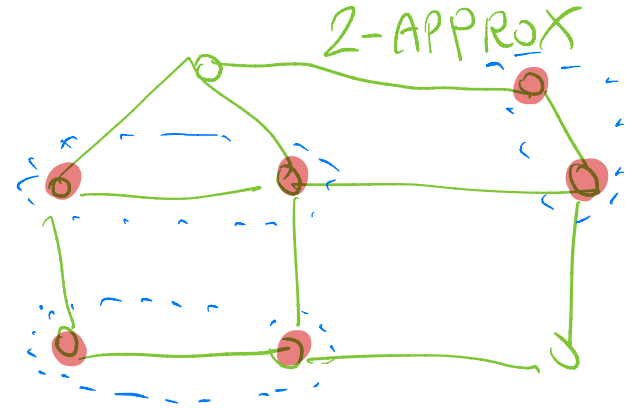
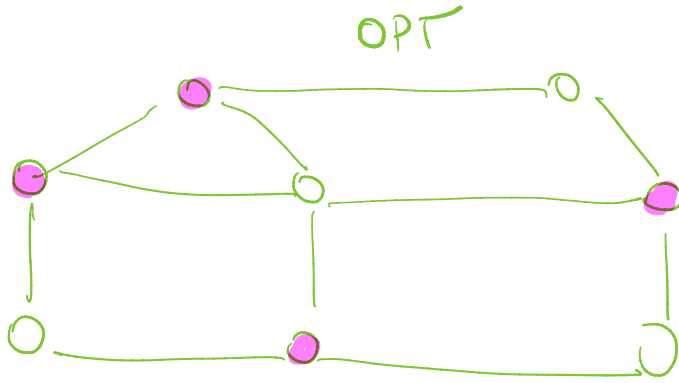
We did not see Linear Programming (LP) techniques as their topic is more suitable for Operation Research classes

PROBLEM	APPROX. FACTOR	LIMIT TO APPROX
Knapsack	$(1-\epsilon)$ FPTAS	—
MIN VERTEX COVER	2	≥ 1.36 (NO FPTAS)
MIN SET COVER	$O(\lg n)$	$\Omega(\lg n)$
MAX INDEPENDENT SET	$O(n)$	$\Omega(n^{1-\epsilon})$ for any $\epsilon > 0$
TSP	NP-hard	NO APPROX

These are situations that can occur

VERTEX COVER (VC)

$G = (V, E)$: $S \subseteq V$ is vertex cover iff $\forall uv \in E : u \in S \text{ or } v \in S$



S is min VC if $\nexists |S'| < |S|$ s.t. S' is also VC

2-approx :

$S = \{\}$

for each edge $uv \in E$:

if both $u, v \notin S$ then $S = S \cup \{u, v\}$

$OPT \geq LB$

$LB = \frac{|E|}{2}$ as any two edges added to S have no common node

OPT must choose at least one node per added edge in S

SET COVER :

universe U , collection of sets $C \subseteq 2^U$

$C' \subseteq C$ is a set cover iff $\bigcup_{S \in C'} S = U$

min set cover C' if no C'' exists $|C''| < |C'|$ and C'' is a set cover

$\uparrow$
sets

$O(\lg n)$ -approx:

$C' = \{\}$

while $\exists x \in U$ s.t. $x \notin \bigcup_{S \in C'} S$ (not covered) do:

choose $S \in C \setminus C'$ s.t. $S \cap (U \setminus \bigcup_{S \in C'} S)$ is maximized

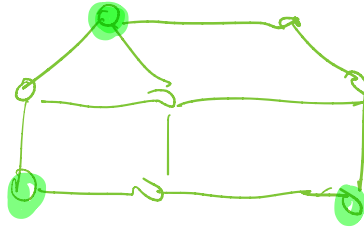
$C' = C' \cup S$



INDEPENDENT SET (IS)

$$G = (V, E)$$

$S \subseteq V$ is an independent set iff $\forall u, v \in V: uv \notin E$



max IS : no S' s.t. $|S'| > |S|$ and S' is an IS

(note: S is IS iff $V \setminus S$ is a vertex cover)

Still, an independent research direction asked: what if we want to solve an NP-hard problem exactly? Ok, we can only find exp time unless $P=NP$... but we can lower the cost with a better exp. time?

PARAMETERIZED ALGORITHMS

GOAL: Complexity $O(f(k) \cdot \text{poly}(n))$ time

k è un [†] parametro
significativo del problema

VERTEX COVER

$$\left. \begin{array}{l} - G=(V,E) \\ - k \geq 1 \end{array} \right\} \exists \text{ VC } S \text{ of } G \text{ s.t. } |S| \leq k$$

k is not necessarily linked to $n=|V|$ or $m=|E|$

FIRST IDEA

Consider all subsets $S \subseteq V$ s.t. $|S| \leq k$ and check if any S is VC

Not parameterized as there are $O\left(\binom{n}{k}\right)$ subsets S to check.

Even though each check requires $\text{poly}(n)$ time, the total

cost is still $O\left(\binom{n}{k} \text{poly}(n)\right) = O(n^k \cdot \text{poly}(n)) = O(n^{k+O(1)})$

Not fitting the format $O(f(k) \cdot \text{poly}(n))$

✗ no appearance of n

BETTER IDEA

Use kernelization technique

[1] Graph G with $n = |V|$, $m = |E|$

we obtain G_k with $\leq k^2 + k$ nodes and $\leq k^2$ edges

TRANSFORMATION IS OBTAINED USING SOME NECESSARY CONDITION

now G_k has size $O(k^2)$ and does not depend on n, m
so we can use any exp-time alg. on G_k paying $O(f(k))$
time

[2] G has a VC of size $\leq k$ iff G_k has a VC of size $\leq k$

Before discussing the kernelization of VC, let us see some "easy" cases.

Easy case 1: max degree in G is 1

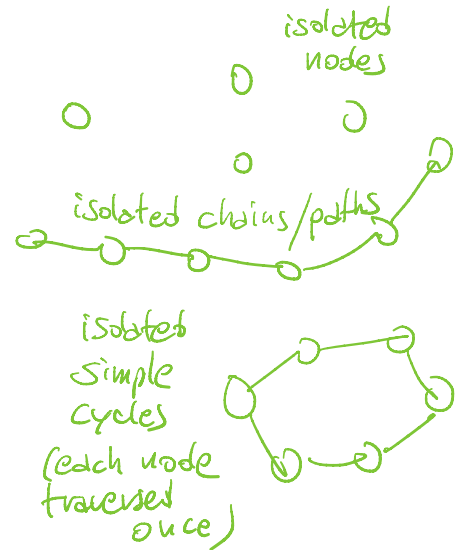
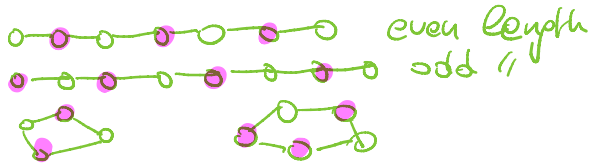


We can only have isolated nodes or edges: optimal VC is obtained by choosing one node per edge

Easy case 2: max degree in G is 2

optimal solution:

- discard isolated nodes
- take half of the nodes in paths, cycles



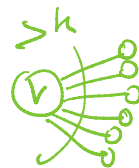
isolated
simple
cycles
(each node
traversed
once)

KERNELIZATION FOR VERTEX COVER : $G \rightsquigarrow G_k$

We recursively apply the first two rules on the current $\langle G, k \rangle$

[1] If v is an isolated node in G , solve $\langle G - \{v\}, k \rangle$

[2] If v is a node of degree $> k$, then v must belong to all VCs of size $\leq k$
Solve $\langle G - \{v\}, k-1 \rangle$



When [1] and [2] cannot be applied anymore:

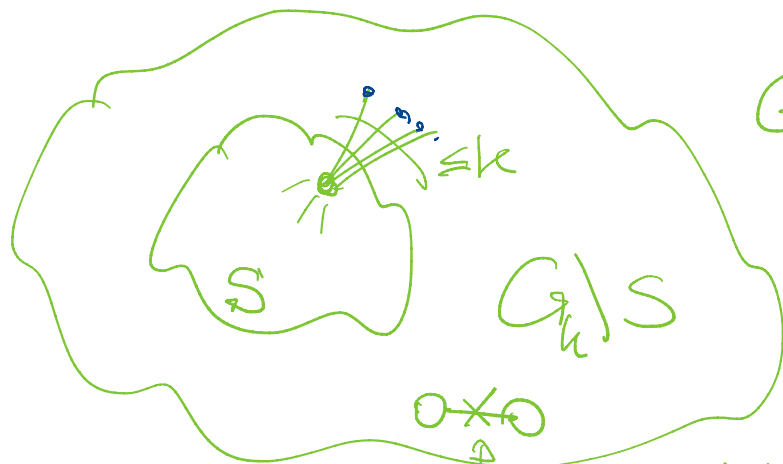
[3] if $k < 0$ or G has $> k^2 + k$ nodes or G has $> k^2$ edges
there is no solution, i.e. no VC of size $\leq k$ in G

→ see next

When ① and ② are no more satisfied, we have a connected graph whose nodes have all degree $\leq k$. Let G_k be this graph.

Prop If G_k has a VC of size $\leq k$ then it has $\leq k^2 + k$ nodes and $\leq k^2$ edges.

Let S be a VC of size $\leq k$ in G_k



G_k has $\leq |S| \cdot k$ edges, $|S| \leq k$
 $\Rightarrow \leq k^2$ edges

$$\begin{aligned} \text{has } |V[G_k \setminus S]| + |S| &\leq \\ &\leq |S| \cdot k + k = k^2 + k \end{aligned}$$

impossible as otherwise S would not be a VC

KERNELIZATION BY BRANCHING

Instance $\langle G, k \rangle$

Let v be a max-degree node

Let $N(v) = \{v' \in V \mid (v, v') \in E\}$ be the set of neighbors of v
(recall that the degree of v is $|N(v)|$)



If the degree of v is 1 then we have easy case 1
and we can find optimal solution in poly time

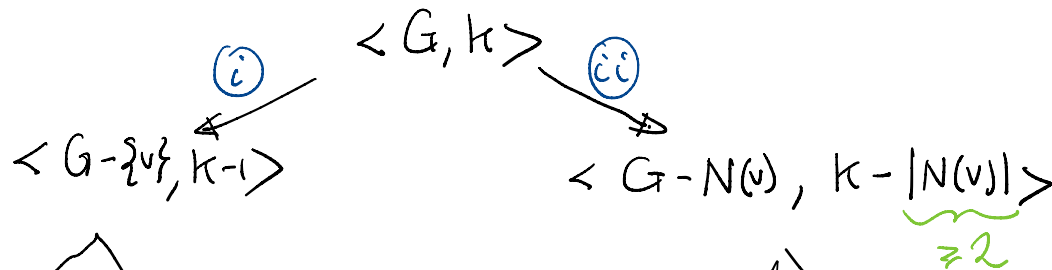
0.0000 $\longrightarrow$

Consider any VC, and recall that $|N(v)| \geq 2$

(i) either v belongs to the VC (and we do not know for $N(v)$)

(ii) or all of $N(v)$ should belong to the VC (as v is not part of it)

Look at the recursion tree:



Recursion
continues here

Recursion
continues here

In the recursion, we stop and say YES when G is empty and $k \geq 0$

Each recursion call takes $\text{poly}(n)$ time $\Rightarrow$

$$\text{Total cost} = O(\# \text{ recursive calls} \cdot \text{poly}(n)) = O(\# \text{ leaves} \cdot \text{poly}(n))$$

as the recursion tree has
always two children, so its total
number of nodes (i.e. recursive calls)
is at most twice the number of leaves

Let $L(k)$ be the number of leaves descending from call $\langle G, k \rangle$

$$L(k) = \begin{cases} 1 & \text{if } k=1 \\ L(k-1) + L(k - \underbrace{|N(u)|}_{\geq 2}) & \end{cases} \leq L(k-1) + L(k-2)$$

$$L(k) \leq O(\Phi^k) \text{ where } \Phi = 1.6\dots \text{ golden ratio}$$

⚡ FIBONACCI!

Total cost is $O((1.6\dots)^k \cdot \text{poly}(n))$

We can do better as we know how to find the optimal solution when max degree is 2 (easy case 2).

$$\text{Hence } |N(v)| \geq 3 \Rightarrow L(k) \leq L(k-1) + L(k-3)$$
$$\sum L(k) = O(1.46^k)$$

Total cost is now $O((1.46)^k \text{poly}(n))$

We can do better! Using the first kernelization in $\text{poly}(n)$ time, we obtain G_k : just use the above algorithm on G_k !

Total cost is further reduced to

$$O(f(k) + \text{poly}(n)) = O((1.46)^k \cdot \text{poly}(k) + \text{poly}(n))$$

run the above alg. on G_k $\nearrow$ obtain G_k